PROTOFIRE

January 2026

Smart contract audit
rain.merkle

Content

Project description

The repository `rain.merkle` implements a Rainlang extension that exposes Merkle tree verification logic to the Rain interpreter. It wraps the standard OpenZeppelin 5.x MerkleProof library into a single Rainlang word: merkle-proof-verify.

The extension allows Rain expressions to verify that a specific leaf node belongs to a Merkle tree defined by a given root. The merkle-proof-verify opcode is variadic; it accepts a dynamic number of inputs where the first argument is the Merkle root, the second is the leaf node to verify, and the remaining arguments constitute the proof sibling path.

Executive summary

Type	Library
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	README.md
Repositories	https://github.com/rainlanguage/rain.merkle

Reviews

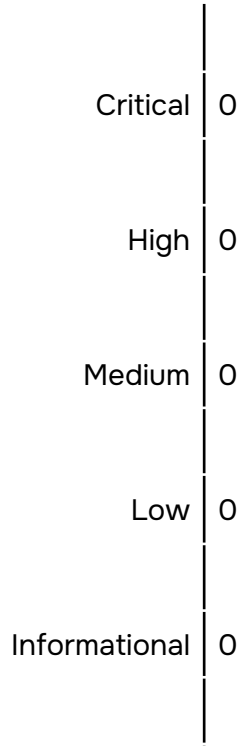
Date	Repository	Commit
28/01/26	rain.merkle	4d89f934342367f5be6ac4e03fd86d3d765dcff7

Scope

Contracts
src/abstract/MerkleExtern.sol
src/abstract/MerkleSubParser.sol
src/concrete/MerkleWords.sol
src/lib/op/LibOpMerkleProofVerify.sol

src/lib/parse/LibMerkleSubParser.sol
src/generated/MerkleWords.pointers.sol

Technical analysis and findings



Security findings

**** Critical

No critical severity issue found.

*** High

No high severity issue found.

** Medium

No medium severity issue found.

* Low

No low severity issue found.

Informational

No informational severity issue found.

General Risks

- Boolean Return Value: The merkle-proof-verify opcode returns 1 (true) for a valid proof and 0 (false) for an invalid one; it does not revert execution automatically. Rainlang authors must explicitly handle the return value (e.g., by using the ensure opcode) to enforce validity. Failure to do so will result in the expression continuing execution with an invalid proof state.

Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating
Methodology:

	**Likelihood** Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	**Impact** Measures the technical loss and business repercussions resulting from a successful attack
	**Severity** Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	**Advanced DeFi Scrutiny** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	**Semantic Consistency Checks** We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
	**Security Analysis** The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.